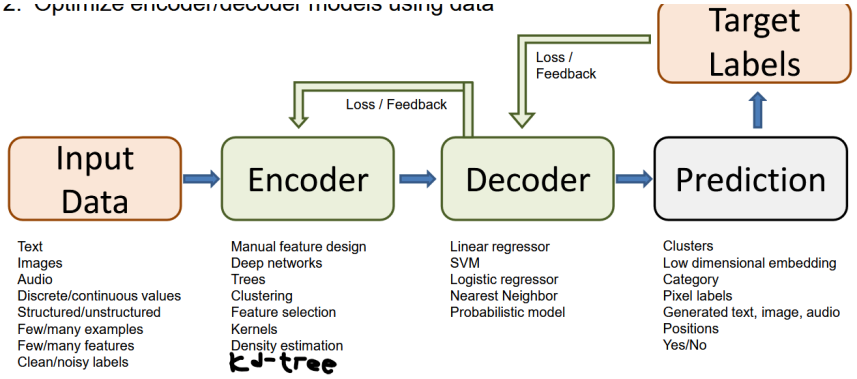
ML

1. The whole machine learning problem
    1. Data preparation
        1. Collect and curate data
        2. Annotate the data (for supervised problems)
        3. Split your data into train, val, test sets
    2. Algorithm and model development
        1. Design methods to extract features from the data
        2. Design machine learning model and identify key parameters and loss
        3. Train, select params, and evaluate your designs using the val set
    3. Final evaluation using test set
    4. Integrate into your application
2. Optimize encoder/decoder models using data



3. Supervised learning, regression
    1. $f(X) = P(Y|X)$
4. dataset X, marginal distribution, $\mu(x) = P(X)$
5. defined so called regression function, $\eta(x) = E[Y|X = x]$, read: expectation of the prediction Y given $X = x$.
6. In the special case of classification, the regression function can be rewritten as $\eta(x) = 0 \cdot P(Y = 0, X = x) + 1 \cdot P(Y = 1|X = x) = P(Y = 1|X = x)$, but this is just regression! or being reduced to a regression problem.
7. If the expectations of the predictions, $\eta(x)$ is close to 0.5, i.e. if η is close to 1, then classifying x is easy if η is close to 0.5, then classifying x is difficult.
8. The probability distribution $P(X, Y)$ is uniquely determined by μ and η.
9. Intuition (discrete case): We can rewrite $P(X = x, Y = 1) = P(Y = 1|X = x)P(X = x) = \eta(x)\mu(x)$ and similarly, $P(X = x, Y = 0) = P(Y = 0|X = x)P(X = x) = (1 - \eta(x))\mu(x)$
10. So we can express the probability of any event (x, y) in terms of η and μ.
11. For formal proof for the general case: see the book of Devorye, Gyorfi, Lugosi, first pages.
12. Explicit form of Bayes classifier under 0-1 loss.
    1. Note: now assume the classifier $f(X) = P(Y|X = x)$ gives either 0 or 1 or $\{0, 1\}$.
    2. The risk of a classifier under the 0-1 loss counts how often the classifier fails, that is $R(f) = E(l(X, Y, f(X))) = E(\mathbf{1}_{f(X)\neq Y}) = P(f(X) \neq Y)$.
    3. The bayes classifier f^* was defined as the classifier that minimizes the **true** risk. This is a hypothetical classifier function.
13. Note: Linear regression
    1. Why not just use matrix inversion? because matrix inversion scales badly with number of features d. With n rows and d features compute is O(d^3 + nd^2) and overhead space is O(d^2). Iterative (gradient descent) methods are better for larger datasets. For small d matrix inversion scales just as well as gradient descent.
14. Empirical Risk Minimization (ERM)
    1. Define a set $\mathcal{F}$ of functions from $X \to Y$.
    2. Within these functions, choose one that has the smallest empirical risk.

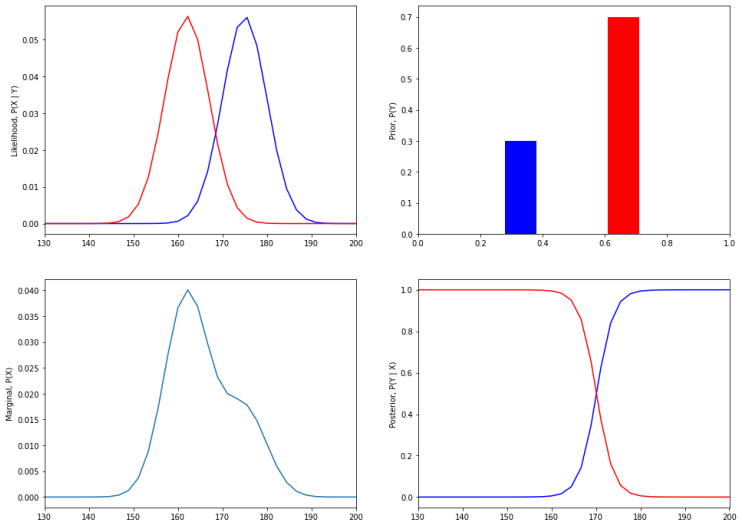
        1. $f_n = \arg\min_{f\in\mathcal{F}} R_n(f)$

    2. (if minimum does not exist, only infimum exists, then it will be that converges to a function f_n. The function sequence classifier functions may not even be unique, but in practice it does not pose any problems).
3. Estimation vs approximation (model) error
4. Underfitting means $\mathcal{F}$ is too small:
    1. small estimation error.
    2. large model error (approximation error).
5. Overfitting means $\mathcal{F}$ is too large:
    1. large estimation error.
    2. small model error (approximation error).
6. Bias-variance trade-off
    1. Bias term: how much difference between f_n and f^* (same flavour as model error).
    2. Variance term: the variance of the RV f_n (estimation error).
7. ERM is a straightforward learning principle.
8. The key to success/failure of ERM is to choose a 'good' function class $\mathcal{F}$.
9. Finding the minimizer of the 0-1 loss can be challenging as it is often NP-hard. In practice, we use convex relaxation of the 0-1 loss function, see below.
10. Approach:
    1. Define regularized risk: $R_{reg,n}(f) = R_n(f) + \lambda\Omega(f)$. Here $\lambda > 0$ is called a regularizer constant.
    2. Then choose $f \in \mathcal{F}$ to minimize the regularized risk.
    3. Regularizer, $\Omega : \mathcal{F} \to \mathrm{R}_{>0}$, that measures how 'complex' a function is,
    4. Examples:
        1. $\mathcal{F} = \text{polynomials}$, $\Omega f = \text{degree of the polynomial f}$.
        2. $\mathcal{F} = \text{differential functions}$, $\Omega(F) = \text{maximal slope}$.
    5. if λ very small: overfitting, high bias error.
    6. if λ very large: underfitting, high variance error.
15. when we talk about a distribution
    1. model
    2. parameter
    3. random variable
16. distribution models (based on study order):
    1. Bernoulli, RV: $X = \begin{cases} 1, \text{ with probability p} \\ 0, \text{ with probability q=1-p} \end{cases}$
    2. Binomial, RV: getting exactly k successes in n independent trials.
    3. Normal/Gaussian, RV: continuous real number.
    4. Poisson, RV: number of events in a fixed interval of time (or space).
    5. Exponentials, RV: the amt of time until a next event occurs (success).
    6. Gamma, RV: the wait time until the k-th event occurs.
    7. Chi-squared, RV: sum of squared of independent standard normal RVs.
    8. Beta, RV: continuous value between 0 and 1, i.e. $[0, 1] \in \mathrm{R}$.
17. Bayesian decision rule

    1. Given the height of a person we want to predict the gender.
    2. RV: Y: male or female, RV: X: height.
    3. Code:

```
# -*- coding: utf-8 -*-
"""
Created on Sat Apr 11 20:17:07 2026

@author: reina
"""

import numpy as np
import matplotlib.pyplot as plt

# probability density function or likelihood
def gaussian_pdf(x, mu, var):
a = 1 / np.sqrt(2 * np.pi * var)
b = - (x - mu)**2 / var
return a * np.exp(b)

# RV: X: height
```

```
# RV: Y: male or female
# want to predict P(Y | X)

# Define likelihood
x = np.linspace(0, 220, num=100)
mu1 = 175; var1 = 50
mu2 = 162; var2 = 50
likelihood1 = gaussian_pdf(x, mu1, var1)
likelihood2 = gaussian_pdf(x, mu2, var2)

# Define priors
prior1 = 0.3
prior2 = 1 - prior1

# Compute marginal
marginal = likelihood1 * prior1 + likelihood2 * prior2

# Compute posteriors
posterior1 = likelihood1 * prior1 / marginal
posterior2 = likelihood2 * prior2 / marginal

fig = plt.figure(figsize=(16, 12))

# Plot likelihood
ax1 = fig.add_subplot(221)
ax1.plot(x, likelihood1, color='b')
ax1.plot(x, likelihood2, color='r')
ax1.set_xlim(130, 200)
ax1.set_ylabel('Likelihood, P(X | Y)')

# Plot priors
ax2 = fig.add_subplot(222)
ax2.bar(0.33, prior1, width=0.1, color='b')
ax2.bar(0.66, prior2, width=0.1, color='r')
ax2.set_xlim(0, 1)
ax2.set_ylabel('Prior, P(Y)')

# Plot marginal
ax3 = fig.add_subplot(223)
ax3.plot(x, marginal)
ax3.set_xlim(130, 200)
ax3.set_ylabel('Marginal, P(X)')

# Plot posterior
ax4 = fig.add_subplot(224)
ax4.plot(x, posterior1, color='b')
ax4.plot(x, posterior2, color='r')
ax4.set_xlim(130, 200)
ax4.set_ylabel('Posterior, P(Y | X)')
```



Logistic regression

1. Steps:
    1. compute distance to the hyperplane via dot product, $d = w^T x$, since w is the vector normal to the hyperplane.
    2. pass the distance through a sigmoid function that outputs 0 or 1.
    3. compute loss using cross-entropy loss or binary cross entropy loss.
2. Although the sigmoid function is nonlinear, the classification itself is a linear operation since it has a linear hyperplane.

K-means:

1. Special case of EM, special case of Autoencoder
2. discrete distribution

References:

1. CS441 UIUC, Derek Hoeim, 2025.
2. Statistical Machine Learning, Tubingen, 2024.